endava

UNIVERSITATEA DIN BUCUREȘTI
VIRTUTE ET SAPIENTIA

PROGRAMARE ORIENTATA PE OBIECTE II

CURS 5

AGENDA CURS 5

1. Managementul situatiilor neprevazute: exceptii

EXCEPTII

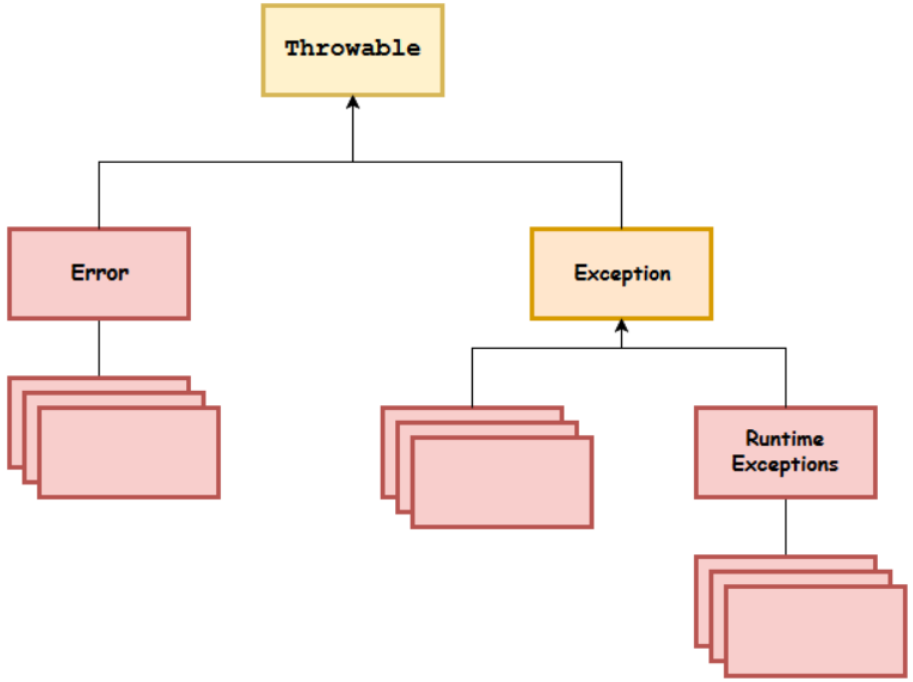
- Java foloseste mecanismul de exceptii pentru a trata situatii neasteptate intr-un mod controlat.
- Situatiile neprevazute sunt de doua tipuri:
  - Erori: aplicatia nu isi poate reveni dupa ce o eroare s-a intamplat
  - Exceptii: aplicatia isi poate continua executia dupa ce o exceptia s-a intamplat, daca acea exceptie a fost tratata in cod (aplicatia stie sa execute o anumita logica in cazul in care apare o anumita exceptie)

Murphy's law: "Whenever you need a crucial file from disk, it will not be there".

endava

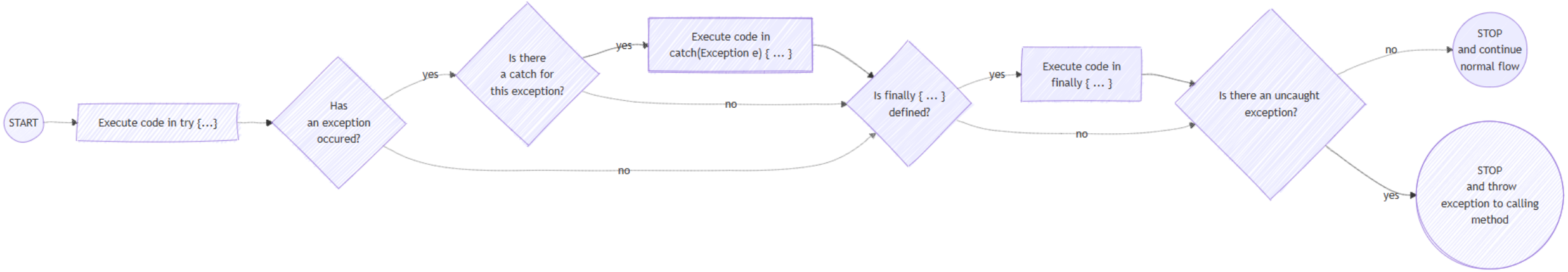
EXCEPTII

- **Error**: acestea sunt conditii anormale, dincolo de controlul programatorului. Nu ar trebui niciodata sa incerci sa tratezi o eroare in vreun fel.
- **Exception**: clasa parinte pentru toate exceptiile existente in lume, care la randul ei are doua subclase:
  - **Checked exceptions**: acestea sunt folosite, de obicei, atunci cand vrei sa obligi programatorul care foloseste API-ul tau sa se gandeasca la modul in care sa le trateze. Aceste tipuri de exceptii sunt verificate la compilare.
  - **Runtime exceptions** sau **unchecked exceptions**. Sunt folosite atunci cand situatia exceptionala nu poate fi recuperata si nu poti face nimic in legatura cu asta. Programatorul nu este obligat sa trateze aceste tipuri de exceptii.



EXCEPTII

1. Tratarea exceptiilor cu try – catch – finally



EXCEPTII

- try nu poate exista singur
  - ✅ Valid: try + catch, try + finally, try + catch + finally
  - ❌ Invalid: doar try, fara catch sau finally
- Ordinea blocurilor este fixa: try → catch(optional) → finally(optional)
  - ❌ catch inainte de try – eroare de compilare
  - ❌ finally inainte de catch – eroare de compilare
- Pot exista mai multe blocuri catch. Se folosesc pentru a trata diferit tipuri diferite de exceptii.
- Pentru a trata tipuri diferite de exceptii in acelasi mod, se foloseste multi-catch. Tipurile de exceptii se separa prin |, trebuie sa nu fie polimorfice, iar referinta este definita o singura data.
  - ✅ catch(SQLException | IOException | JSONFormatException e)
- Blocurile catch trebuie sa fie in ordine de la cele mai specifice la cele mai generale.
  - ❌ catch(Exception e) inainte de catch(FileNotFoundException e) – va da eroare de compilare

endava

EXCEPTII

- Exceptia prinsa este disponibila doar in blocul catch.
- Poti arunca o noua exceptie din catch sau finally, cu **throw**
  - throw new IllegalArgumentException()
- finally se executa intotdeauna, indiferent daca se arunca sau nu o exceptie.
- finally poate fi omis doar daca folosesti try-with-resources (resursele se inchid automat).
- Daca finally contine return, acesta suprascrie orice alt return.

endava

EXCEPTII

Best practices:

- Nu lasa blocuri catch goale
- Logheaza corect exceptiile, fara mesaje vagi
- Prinde exceptii specifice, nu generice. Astfel tratezi exact cauza problemei si nu ascunzi alte erori neprevazute.
- Creeaza exceptii custom doar daca e necesar, si defineste-le ca subtipuri de RuntimeException
- Nu folosi exceptii pentru controlul normal al fluxului

❌ Gresit

```
try {
    int x = list.get(index);
} catch (IndexOutOfBoundsException e) {
    // logica alternativa
}
```

✅ Corect

```
if (index >= 0 && index < list.size()) {
    int x = list.get(index);
}
```


endava

EXCEPTII

```java
public void loadConfig() {
    try {
        FileReader reader = new FileReader("config.txt");
        // citire configuratie...
    } catch (Exception e) {

    }
}
```

Probleme:

- Prinde exceptii generice (Exception)
- Nu se ofera niciun mesaj de eroare sau log
- Nu se face inchiderea resursei
- Daca fisierul nu exista sau e corupt, nu stii de ce nu functioneaza aplicatia

endava

EXCEPTII

2. Propagarea exceptiilor este mecanismul prin care o metoda nu trateaza local o exceptie, ci o transmite mai departe (catre metoda apelanta), folosind cuvantul cheie **throws.**

- Cand folosesti propagarea?
  - Cand o metoda nu stie sau nu trebuie sa trateze o exceptie.
  - Cand vrei ca responsabilitatea tratarii sa fie in alta parte

```
public void citesteFisier() throws FileNotFoundException {
    FileReader fr = new FileReader("fisier.txt");
}
```

```
public static void main(String[] args) {
    try {
        new Clasa().citesteFisier();
    } catch (FileNotFoundException e) {
        System.out.println("Fisierul nu a fost gasit.");
    }
}
```

EXCEPTII

2. Propagarea exceptiilor

- Exceptii checked trebuie fie tratate (try-catch), fie propagate (throws) (ex: IOException, SQLException)
- Exceptii de runtime (unchecked) nu sunt obligate sa fie tratate sau propagate (ex: NullPointerException, IllegalArgumentException).

```java
public class ExceptionPropagationExample {

    public static void main(String[] args) {
        try {
            readData();
        } catch (Exception e) {
            System.out.println("Exceptie prinsa in main: " + e.getMessage());
        }
    }

    // Metoda care propaga exceptia catre apelant
    public static void readData() throws Exception {
        System.out.println("Incearca sa citesti date...");
        // Simulam o eroare
        throw new Exception("Eroare la citirea datelor.");
    }
}
```

EXCEPTII

2. Propagarea exceptiilor

- O metoda suprascrisa poate arunca:
  - ✅ aceeasi exceptie ca metoda din clasa de baza
  - ✅ o subclasa a exceptiei aruncate de metoda originala
  - ❌ nu poate arunca o exceptie mai generala
  - ❌ nu poate arunca o exceptie checked complet noua (care nu exista in semnatura metodei de baza).
  - ✅ poate arunca o exceptie de runtime complet noua

EXCEPTII

2. Propagarea exceptiilor

```java
// Clasa de baza
class Calculator {
    public void calculate() throws Exception {
        System.out.println("Calcul de baza...");
    }
}

// Clasa derivata
class AdvancedCalculator extends Calculator {
    @Override
    public void calculate() throws ArithmeticException {
        System.out.println("Calcul avansat...");
        // Simulam o exceptie
        throw new ArithmeticException("Impartire la zero.");
    }
}
```

```java
// Clasa principala
public class Main {
    public static void main(String[] args) {
        Calculator calc = new AdvancedCalculator();

        try {
            calc.calculate();
        } catch (ArithmeticException e) {
            System.out.println("Exceptie aritmetica: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Alta exceptie: " + e.getMessage());
        }
    }
}
```

Q&A

THANK YOU

DANIELA SPILCĂ